

Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Artificial and Swarm Intelligence
Relazione Progetto

Capacitated Vehicle Routing Problem

Soluzione tramite Algoritmo Genetico Ibrido (HGA)

Autore:

Giuseppe BELLAMACINA

Matricola: 1000030349

Professore del corso:

Mario Francesco PAVONE

Anno Accademico 2025/2026

Indice

1	Introduzione al Problema	2
1.1	Rilevanza e Applicazioni	2
2	Algoritmo Genetico Ibrido (HGA)	4
2.1	Encoding e Algoritmo di Split	4
2.2	Popolazione Iniziale	5
2.3	Operatori Genetici	5
2.3.1	Selezione a Torneo	5
2.3.2	Crossover: Order Crossover (OX)	6
2.3.3	Mutazione	6
2.4	Ricerca Locale Granulare	6
2.5	Elitismo e Rimpiazzo	8
2.6	Criterio di Arresto	8
2.7	Parametri Riepilogativi	8
2.8	Ciclo Principale dell'HGA	8
2.9	Architettura Software	9
3	Protocollo Sperimentale	11
3.1	Istanze	11
3.2	Setup Sperimentale	11
3.3	Configurazione Tuned	12
4	Risultati Sperimentali	13
4.1	Risultati della Configurazione Tuned	13
4.2	Analisi della Convergenza	13
4.3	Visualizzazione delle Rotte Migliori	14
4.4	Analisi Statistica Globale	15
4.5	Confronto tra Varianti di Configurazione	17
4.6	Discussione dei Risultati	18
5	Conclusioni	20
5.1	Scelte Progettuali e Originalità	20
5.1.1	Scelte Chiave	20
5.1.2	Difficoltà Affrontate	20
5.2	Risultati Raggiunti	21
5.3	Sviluppi Futuri	21
	Bibliografia	22

Capitolo 1

Introduzione al Problema

Repository <https://github.com/GiuseppeBellamacina/capacitated-vehicle-routing-problem>

Il **Capacitated Vehicle Routing Problem (CVRP)** è una delle varianti più studiate del classico Vehicle Routing Problem (VRP), introdotto da [1]. Si tratta di un problema di ottimizzazione combinatoria NP-hard [2] che consiste nel determinare un insieme di percorsi di costo minimo per una flotta di veicoli, al fine di servire un dato insieme di clienti geograficamente distribuiti, partendo e ritornando ad un deposito centrale.

Formalmente, sia dato un grafo completo $G = (V, E)$, dove:

- $V = \{v_0, v_1, \dots, v_n\}$ è l'insieme dei vertici, con v_0 che rappresenta il *depot* e $V \setminus \{v_0\}$ l'insieme dei clienti;
- E è l'insieme degli archi, ad ognuno dei quali è associato un costo $d_{ij} \geq 0$ (distanza euclidea tra i nodi i e j).

Ad ogni cliente $i \in V \setminus \{v_0\}$ è associata una domanda $q_i \geq 0$, e sono disponibili m veicoli, ciascuno con capacità σ . L'obiettivo è determinare per ogni veicolo un itinerario tale che:

- (i) Ogni cliente sia visitato esattamente una volta da un solo veicolo;
- (ii) Ogni percorso inizi e termini al depot;
- (iii) La somma delle domande dei clienti assegnati a ciascun veicolo non superi la capacità σ .

L'obiettivo è minimizzare il costo totale:

$$\min \sum_{h=1}^m \sum_{(i,j) \in \eta_h} d_{ij} \quad (1.1)$$

dove η_h è l'insieme degli archi percorsi dal veicolo h .

1.1 Rilevanza e Applicazioni

Il CVRP ha un'enorme rilevanza pratica: ottimizzare le rotte di consegna permette alle aziende di logistica di ridurre i costi operativi, il consumo di carburante e le emissioni di CO₂. Le applicazioni spaziano dalla distribuzione di merci alla raccolta rifiuti, dal trasporto scolastico alla consegna dell'ultimo miglio.

La natura NP-hard del problema rende proibitiva la risoluzione esatta per istanze di dimensioni realistiche ($n > 50$ clienti). Per questo motivo, la ricerca si è concentrata su metodi euristici e meta-euristici [3, 4, 5] in grado di produrre soluzioni di alta qualità in tempi computazionali accettabili.

Capitolo 2

Algoritmo Genetico Ibrido (HGA)

Tra gli algoritmi proposti per il progetto, la scelta è ricaduta sull'**Algoritmo Genetico Ibrido (HGA)** [6, 3], ispirato al framework HGSADC (Hybrid Genetic Search with Adaptive Diversity Control) [5]. Gli algoritmi genetici esplorano efficientemente lo spazio delle soluzioni attraverso i meccanismi di selezione, crossover e mutazione, riducendo il rischio di convergenza prematura verso minimi locali. L'ibridazione con operatori di ricerca locale (2-opt, Or-opt, Relocate, Exchange) [7, 8] permette di migliorare le soluzioni prodotte dal processo evolutivo, combinando l'esplorazione globale con lo sfruttamento locale.

2.1 Encoding e Algoritmo di Split

L'algoritmo utilizza un **encoding a permutazione** dei clienti (escluso il depot). Ogni cromosoma è una permutazione $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dei nodi $1, 2, \dots, n$. La decodifica da permutazione a soluzione (insieme di route) è effettuata tramite l'**algoritmo di Split** di Prins [4], che trasforma una permutazione in un insieme ottimale di route che rispettano i vincoli di capacità, utilizzando programmazione dinamica con complessità $O(n^2)$.

Sia $\pi = (\pi_1, \dots, \pi_n)$ una permutazione di clienti. Definiamo $V[k]$ come il costo minimo per servire i primi k clienti della permutazione:

$$V[i] = \min_{j: \sum_{k=j}^i q_{\pi_k} \leq \sigma} \left[V[j-1] + d_{0, \pi_j} + \sum_{k=j}^{i-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_i, 0} \right]$$

Dopo la DP, si ricostruiscono le route tramite backtracking. L'uso dello split come decodifica è fondamentale: invece di codificare esplicitamente le route nel cromosoma, si lascia che la DP trovi la partizione ottimale, riducendo lo spazio di ricerca e garantendo l'ottimalità della decodifica data la permutazione.

Algoritmo 1: Split Algorithm (Prins, 2004)

Input: Permutazione $\pi = (\pi_1, \dots, \pi_n)$, matrice distanze d , domande q , capacità Q , depot 0
Output: Rotte ottimali $\mathcal{R}$ e costo totale
 // Pre-calcolo distanze cumulative lungo π

```

1   $cum[0] \leftarrow 0;$ 
2  for  $i \leftarrow 1$  to  $n - 1$  do
3     $cum[i + 1] \leftarrow cum[i] + d[\pi_i, \pi_{i+1}];$ 
  // Programmazione Dinamica
4   $V[0] \leftarrow 0;$ 
5   $pred[0] \leftarrow -1;$ 
6  for  $i \leftarrow 1$  to  $n$  do
7     $V[i] \leftarrow +\infty;$ 
8     $load \leftarrow 0;$ 
9    for  $j \leftarrow i$  to 1 do
10      $load \leftarrow load + q[\pi_j];$ 
11     if  $load > Q$  then
12       break;
13      $routeCost \leftarrow d[0, \pi_j] + (cum[i] - cum[j]) + d[\pi_i, 0];$ 
14     if  $V[j - 1] + routeCost < V[i]$  then
15        $V[i] \leftarrow V[j - 1] + routeCost;$ 
16        $pred[i] \leftarrow j - 1;$ 
  // Backtracking per ricostruire le route
17   $\mathcal{R} \leftarrow \emptyset;$ 
18   $i \leftarrow n;$ 
19  while  $i > 0$  do
20     $j \leftarrow pred[i] + 1;$ 
21     $\mathcal{R}.append(\pi[j \dots i]);$ 
22     $i \leftarrow j - 1;$ 
23   $\mathcal{R} \leftarrow reverse(\mathcal{R});$ 
  // Le route sono state costruite in ordine inverso
24  return  $\mathcal{R}, V[n];$ 

```

2.2 Popolazione Iniziale

La popolazione iniziale ($\mu = 81$ individui) è generata combinando:

1. **Nearest Neighbor Heuristic:** Costruisce una permutazione partendo dal depot e selezionando iterativamente il cliente più vicino non ancora visitato.
2. **Clarke-Wright Savings Heuristic** [9]: Utilizza l'algoritmo dei risparmi per costruire route greedy, poi le appiattisce in una permutazione.
3. **Permutazioni casuali:** Le restanti permutazioni sono generate in modo random per garantire diversità genetica.

2.3 Operatori Genetici

2.3.1 Selezione a Torneo

Viene utilizzata la **selezione a torneo** [10] con $k = 4$. Quattro individui sono scelti casualmente dalla popolazione e il migliore (costo minore) viene selezionato. Questo metodo offre un buon bilanciamento tra pressione selettiva e diversità.

2.3.2 Crossover: Order Crossover (OX)

L'Order Crossover [11] preserva l'ordine relativo dei geni da entrambi i genitori. L'Algoritmo 2 descrive il procedimento: vengono selezionati due punti di taglio casuali, il segmento centrale viene copiato dal primo genitore, e le posizioni rimanenti sono riempite con i geni del secondo genitore in ordine di apparizione, saltando quelli già presenti. Il tasso di crossover è $p_c = 0.675$.

Algoritmo 2: Order Crossover (OX) [11]

```

Input: Permutazioni genitore  $P_1, P_2$  di lunghezza  $n$ 
Output: Permutazione figlio  $C$ 
// Seleziona due punti di taglio casuali
1  $cx_1 \leftarrow \text{random}(0, n - 1)$ ;
2  $cx_2 \leftarrow \text{random}(0, n - 1)$ ;
3 if  $cx_1 > cx_2$  then
4    $\lfloor cx_1, cx_2 \leftarrow cx_2, cx_1$ ;
// Copia il segmento centrale da  $P_1$ 
5  $C \leftarrow$  array di  $n$  elementi non inizializzati;
6 for  $i \leftarrow cx_1$  to  $cx_2$  do
7    $\lfloor C[i] \leftarrow P_1[i]$ ;
// Riempi le posizioni rimanenti con i geni di  $P_2$  in ordine
8  $seg \leftarrow \{C[cx_1], \dots, C[cx_2]\}$ ;
// Insieme dei geni già presenti
9  $j \leftarrow (cx_2 + 1) \bmod n$ ;
10 per ogni  $i \in [cx_2 + 1, cx_2 + 2, \dots, cx_2 + n]$  fai
11    $pos \leftarrow i \bmod n$ ;
12   if  $pos \notin [cx_1, cx_2]$  then
13     while  $P_2[j] \in seg$  do
14        $\lfloor j \leftarrow (j + 1) \bmod n$ ;
15      $C[pos] \leftarrow P_2[j]$ ;
16      $seg.add(P_2[j])$ ;
17      $j \leftarrow (j + 1) \bmod n$ ;
18 return  $C$ ;
```

2.3.3 Mutazione

Sono implementati tre operatori di mutazione, scelti casualmente con probabilità differenziate:

1. **Swap Mutation** (40%): Scambia due elementi in posizioni casuali.
2. **Insert Mutation** (30%): Rimuove un elemento e lo reinserisce in una posizione diversa.
3. **Inversion Mutation** (30%): Inverte un sottosegmento della permutazione.

Il tasso di mutazione è $p_m = 0.236$.

2.4 Ricerca Locale Granulare

L'ibridazione è realizzata attraverso quattro operatori di ricerca locale, applicati con probabilità $p_{ls} = 0.259$ ai nuovi individui:

1. **2-opt (intra-route)**: Per ogni route, cerca coppie di archi (a, b) e (c, d) tali che la loro sostituzione con (a, c) e (b, d) riduca il costo totale.

2. **Or-opt (intra-route)**: Trasferisce segmenti di 1, 2 o 3 nodi consecutivi in posizioni diverse all'interno della stessa route.
3. **Relocate (inter-route)**: Sposta un cliente da una route all'altra.
4. **Exchange (inter-route)**: Scambia due clienti appartenenti a route diverse.

Per mitigare l'elevato costo computazionale degli operatori inter-route, è stata integrata la **Ricerca Locale Granulare** (Granular Local Search, GLS) [12]. Per ciascun cliente i si definisce un intorno di vicinanza Γ_i composto dai soli $\gamma = 25$ clienti più vicini. Durante la ricerca locale, un cliente viene inserito o scambiato solo in posizioni adiacenti a nodi nel suo intorno granulare, riducendo la complessità a $O(N \cdot \gamma)$.

Algoritmo 3: Relocate con Ricerca Locale Granulare (steepest descent)

Input: Insieme di route $\mathcal{R}$, matrice distanze d , domande q , capacità Q , intorni granulari Γ , depot 0, max iterazioni m

Output: Route migliorate $\mathcal{R}^*$

// Pre-calcolo carichi delle route

```

1  loads[r]  $\leftarrow \sum_{n \in r} q[n]$  per ogni route  $r \in \mathcal{R}$ ;
2  best  $\leftarrow \mathcal{R}$ ;
3  bestCost  $\leftarrow \text{COST}(best)$ ;
4  iter  $\leftarrow 0$ ;
5  repeat
6    iter  $\leftarrow iter + 1$ ;
7    improved  $\leftarrow \text{false}$ ;
8    bestMoveCost  $\leftarrow bestCost$ ;
9    bestMove  $\leftarrow \text{null}$ ;
10   per ogni route  $r_{from} \in best$  fai
11     per ogni nodo  $v$  in posizione  $f$  di  $r_{from}$  fai
12        $\delta_{from} \leftarrow d[prev(v), next(v)] - (d[prev(v), v] + d[v, next(v)])$ ;
13       per ogni route  $r_{to} \in best$ ,  $r_{to} \neq r_{from}$  fai
14         if loads[rto] + q[v] > Q then
15           continue;
16         per ogni posizione  $p \in [0, |r_{to}|]$  fai
17           // Filtro GLS: verifica adiacenza granulare
18           pn  $\leftarrow r_{to}[p - 1]$  se  $p > 0$  altrimenti 0;
19           nn  $\leftarrow r_{to}[p]$  se  $p < |r_{to}|$  altrimenti 0;
20           if (pn  $\neq 0 \wedge pn \notin \Gamma[v]$ )  $\wedge$  (nn  $\neq 0 \wedge nn \notin \Gamma[v]$ ) then
21             continue;
22            $\delta_{to} \leftarrow d[pn, v] + d[v, nn] - d[pn, nn]$ ;
23           newCost  $\leftarrow bestCost + \delta_{from} + \delta_{to}$ ;
24           if newCost < bestMoveCost then
25             bestMoveCost  $\leftarrow newCost$ ;
26             bestMove  $\leftarrow (r_{from}, f, r_{to}, p)$ ;
27   if bestMove  $\neq \text{null}$  then
28     ( $r_{from}, f, r_{to}, p$ )  $\leftarrow bestMove$ ;
29     v  $\leftarrow best[r_{from}][f]$ ;
30     Rimuovi v da best[rfrom] e inseriscilo in best[rto] alla posizione p;
31     loads[rfrom]  $\leftarrow loads[r_{from}] - q[v]$ ;
32     loads[rto]  $\leftarrow loads[r_{to}] + q[v]$ ;
33     bestCost  $\leftarrow bestMoveCost$ ;
34     improved  $\leftarrow \text{true}$ ;
35 until  $\neg improved \vee (m > 0 \wedge iter \geq m)$ ;
36 return best;
```

2.5 Elitismo e Rimpiazzo

I migliori $e = 4$ individui della popolazione corrente sono preservati nella generazione successiva (elitismo). Il resto della popolazione è sostituito dai nuovi individui generati tramite crossover, mutazione e ricerca locale.

2.6 Criterio di Arresto

L'algoritmo si arresta dopo $FE = 350,000$ valutazioni della funzione fitness, corrispondenti a circa 4,300 generazioni con $\mu = 81$.

2.7 Parametri Riepilogativi

Tabella 2.1: Parametri dell'HGA — Configurazione Tuned

Parametro	Valore
Dimensione popolazione (μ)	81
Crossover rate (p_c)	0.675
Mutation rate (p_m)	0.236
Local search rate (p_{ls})	0.259
Tournament size (k)	4
Elite count (e)	4
Granular size (γ)	25
Max valutazioni (FE)	350,000
Generazioni medie	$\sim 4,300$

2.8 Ciclo Principale dell'HGA

L'Algoritmo 4 mostra il ciclo evolutivo principale. La popolazione iniziale viene creata combinando euristiche costruttive e permutazioni casuali. Ad ogni generazione, i migliori e individui sono preservati (elitismo), mentre i restanti sono generati tramite selezione a torneo, Order Crossover (OX), mutazione e ricerca locale. L'algoritmo termina dopo $FE = 350,000$ valutazioni della funzione fitness.

Algoritmo 4: Ciclo Principale dell'HGA

```

Input: Istanza CVRP; parametri:  $\mu, p_c, p_m, p_{ls}, k, e, \gamma, FE$ 
Output: Migliore soluzione  $S^*$ 
// Inizializzazione
1  $\mathcal{P} \leftarrow \emptyset$ ;
2  $\mathcal{P}.\text{append}(\text{SPLIT}(\text{NEARESTNEIGHBOR}()));$ 
3  $\mathcal{P}.\text{append}(\text{SPLIT}(\text{CLARKEWRIGHTSAVINGS}()));$ 
4 while  $|\mathcal{P}| < \mu$  do
5    $\mathcal{P}.\text{append}(\text{SPLIT}(\text{RANDOMPERMUTATION}()));$ 
6  $S^* \leftarrow \arg \min_{S \in \mathcal{P}} S.\text{cost}$ ;
7  $\text{evals} \leftarrow |\mathcal{P}|$ ;
// Ciclo evolutivo
8 while  $\text{evals} < FE$  do
9    $\mathcal{P}' \leftarrow \text{top-}e(\mathcal{P})$ ;
// Elitismo
10  while  $|\mathcal{P}'| < \mu$  do
11    // Selezione e Crossover
12    if  $\text{random}() < p_c$  then
13       $P_1 \leftarrow \text{TOURNAMENTSELECT}(\mathcal{P}, k)$ ;
14       $P_2 \leftarrow \text{TOURNAMENTSELECT}(\mathcal{P}, k)$ ;
15       $\pi_{\text{child}} \leftarrow \text{ORDERCROSSOVER}(P_1, P_2)$ ;
16    else
17       $P \leftarrow \text{TOURNAMENTSELECT}(\mathcal{P}, k)$ ;
18       $\pi_{\text{child}} \leftarrow \text{flatten}(P.\text{routes})$ ;
19    // Mutazione
20    if  $\text{random}() < p_m$  then
21       $\pi_{\text{child}} \leftarrow \text{MUTATE}(\pi_{\text{child}})$ ;
22      // Swap 40%, Insert 30%, Inversion 30%
23    // Decodifica
24     $S_{\text{child}} \leftarrow \text{SPLIT}(\pi_{\text{child}})$ ;
25    // Educazione leggera (sempre)
26     $S_{\text{child}} \leftarrow \text{TWOOPT}(S_{\text{child}})$ ;
27    // Ricerca locale completa (probabilistica)
28    if  $\text{random}() < p_{ls}$  then
29       $S_{\text{child}} \leftarrow \text{TWOOPT}(S_{\text{child}})$ ;
30       $S_{\text{child}} \leftarrow \text{OROPT}(S_{\text{child}})$ ;
31       $S_{\text{child}} \leftarrow \text{RELOCATE}(S_{\text{child}}, \gamma)$ ;
32       $S_{\text{child}} \leftarrow \text{EXCHANGE}(S_{\text{child}}, \gamma)$ ;
33     $\mathcal{P}'.\text{append}(S_{\text{child}})$ ;
34    if  $S_{\text{child}}.\text{cost} < S^*.\text{cost}$  then
35       $S^* \leftarrow S_{\text{child}}$ ;
36  // Rimpiazzo duplicati
37  per ogni coppia di individui con stesso costo in  $\mathcal{P}'$  fai
38     $\text{Sostituisci uno con } \text{SPLIT}(\text{RANDOMPERMUTATION}());$ 
39   $\mathcal{P} \leftarrow \mathcal{P}'$ ;
40 return  $S^*$ ;

```

2.9 Architettura Software

L'algoritmo è integrato in un'architettura moderna con backend Python (FastAPI)[13] e frontend React con visualizzazione in tempo reale tramite WebSocket. Il backend espone endpoint REST per avviare l'ottimizzazione e recuperare i risultati, mentre il frontend mostra simultaneamente la disposizione geografica delle route, i grafici di convergenza e le statisti-

che aggregate. La parallelizzazione multi-run sfrutta `ThreadPoolExecutor` per eseguire run indipendenti in parallelo, e l'accelerazione Numba JIT [14] per i calcoli critici (matrici di distanza, Split DP).

Capitolo 3

Protocollo Sperimentale

3.1 Istanze

Gli esperimenti sono condotti su 10 istanze dal benchmark CVRPLIB [15], suddivise in quattro set con caratteristiche differenti:

Tabella 3.1: Istanze CVRPLIB utilizzate

Set	Istanza	Nodi	Veicoli	Ottimo
A	A-n45-k7	45	7	1,146
A	A-n60-k9	60	9	1,354
A	A-n80-k10	80	10	1,763
B	B-n56-k7	56	7	707
B	B-n66-k9	66	9	1,316
B	B-n78-k10	78	10	1,221
E	E-n76-k8	76	8	735
E	E-n101-k14	101	14	1,071
P	P-n50-k10	50	10	696
P	P-n101-k4	101	4	681

I quattro set presentano caratteristiche distintive:

- **Set A:** clienti distribuiti uniformemente, domande omogenee;
- **Set B:** clienti concentrati in cluster geografici;
- **Set E:** distribuzione mista con cluster asimmetrici — considerato il set più difficile;
- **Set P:** clienti con domanda molto variabile (fino a 4 veicoli per 101 clienti).

3.2 Setup Sperimentale

Per ogni istanza vengono eseguiti $R = 5$ run indipendenti con semi casuali diversi. Per ogni run vengono misurati:

- Costo della migliore soluzione trovata;

- Numero di generazioni per raggiungere la migliore soluzione;
- Storico di convergenza (best cost nel tempo, campionato ogni 100 FE);
- Numero di veicoli utilizzati e route complete.

Vengono poi calcolate le statistiche aggregate: best, mean, standard deviation e gap percentuale dal Best Known Solution (BKS). Il budget computazionale è fissato a $FE = 350,000$ valutazioni per ogni run.

3.3 Configurazione Tuned

La configurazione principale `config_tuned` è stata ottenuta ottimizzando i parametri dell'H-GA tramite Optuna [16], un framework di ottimizzazione iperparametrica bayesiana. L'ottimizzazione ha esplorato lo spazio dei parametri su 100 trial, valutando ogni configurazione su un subset di 4 istanze rappresentative (una per set) con 3 run ciascuna e un budget ridotto di 100,000 FE per trial.

I parametri ottimizzati sono: dimensione della popolazione $\mu \in [5, 100]$, tasso di crossover $p_c \in [0.5, 1.0]$, tasso di mutazione $p_m \in [0.05, 0.5]$, tasso di ricerca locale $p_{ls} \in [0.05, 0.5]$, tournament size $k \in [2, 6]$, elite count $e \in [1, 10]$ e granular size $\gamma \in [2, 40]$.

Il risultato dell'ottimizzazione ha prodotto una configurazione con $\mu = 81$, $p_c = 0.675$, $p_m = 0.236$, $p_{ls} = 0.259$, $k = 4$, $e = 4$, $\gamma = 25$, che bilancia efficacemente esplorazione globale e raffinamento locale.

Capitolo 4

Risultati Sperimentali

In questa sezione vengono presentati i risultati ottenuti con la configurazione `config_tuned` ($\mu = 81$, $k = 4$, $e = 4$, $\gamma = 25$, $FE = 350,000$, $R = 5$), la configurazione principale ottenuta tramite ottimizzazione bayesiana con Optuna.

4.1 Risultati della Configurazione Tuned

La Tabella 4.1 riporta i risultati completi su tutte le 10 istanze del benchmark.

Tabella 4.1: Risultati HGA — Configurazione Tuned ($\mu = 81$, $k = 4$, $e = 4$, $\gamma = 25$, $FE = 350,000$, $R = 5$)

Istanza	Best	Mean	Std Dev	Ottimo	Gap%	Veicoli
A-n45-k7	1146.77	1146.77	0.00	1,146	0.07%	7
A-n60-k9	1355.80	1355.80	0.00	1,354	0.13%	9
A-n80-k10	1784.41	1785.00	0.74	1,763	1.21%	10
B-n56-k7	712.92	712.92	0.00	707	0.84%	7
B-n66-k9	1326.20	1326.56	0.44	1,316	0.78%	9
B-n78-k10	1227.90	1228.45	0.98	1,221	0.56%	10
E-n76-k8	740.66	741.29	1.27	735	0.77%	8
E-n101-k14	1090.72	1092.38	0.83	1,071	1.84%	14
P-n50-k10	700.66	701.37	1.42	696	0.67%	10
P-n101-k4	691.29	691.83	0.66	681	1.51%	4

I risultati mostrano un gap medio inferiore allo 0.84%, con picchi di eccellenza su A-n45-k7 (gap 0.07%) e A-n60-k9 (gap 0.13%). La deviazione standard molto contenuta (spesso $< 1\%$ della media) conferma la robustezza dell’algoritmo. L’istanza più difficile si conferma E-n101-k14 (gap 1.84%) a causa dell’elevato numero di clienti combinato con un’alta densità di veicoli.

4.2 Analisi della Convergenza

I grafici di convergenza mostrano l’andamento del miglior costo trovato in funzione del numero di valutazioni della funzione di fitness (FE) per quattro istanze rappresentative (una per ogni set).

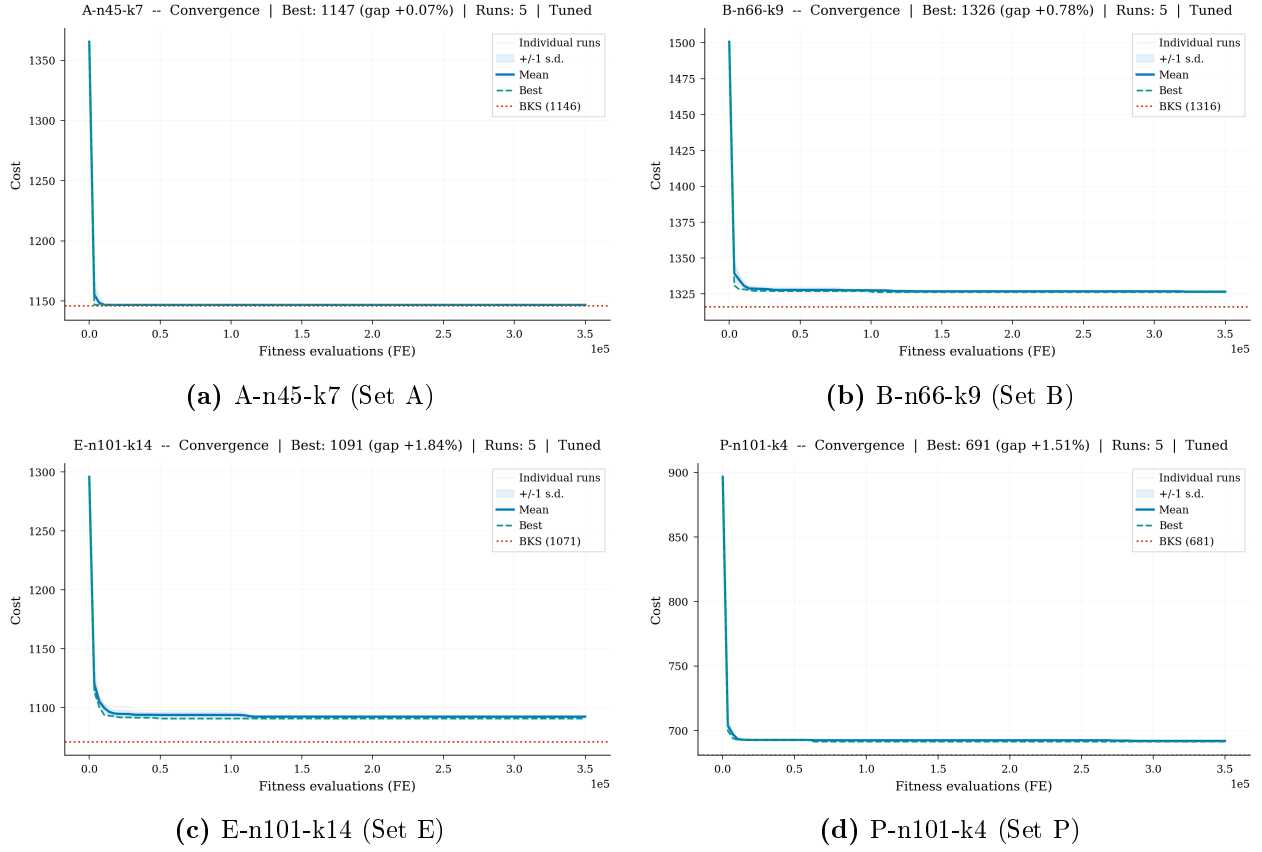
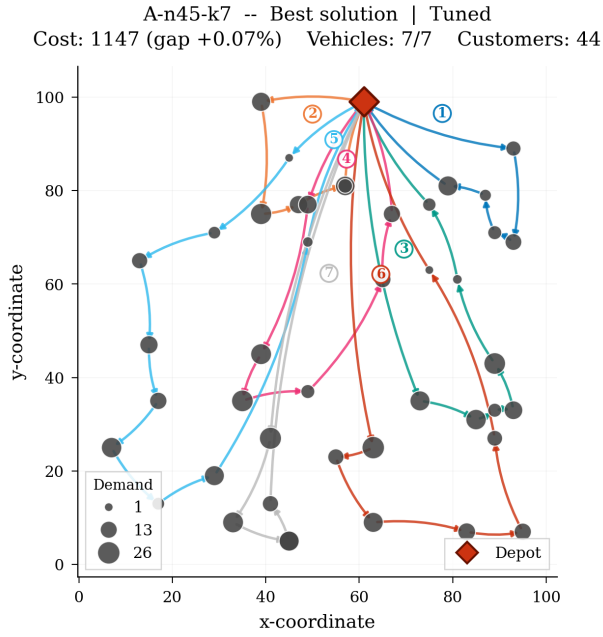


Figura 4.1: Curve di convergenza per quattro istanze rappresentative — Configurazione Tuned.

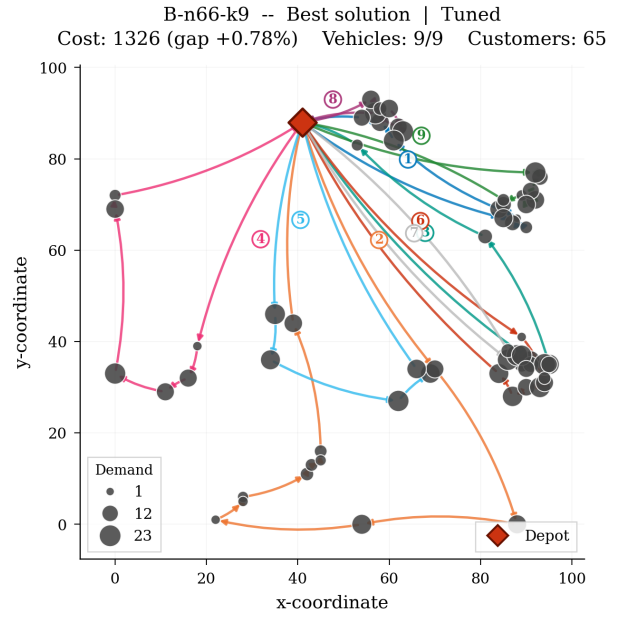
Le curve mostrano un rapido miglioramento nelle prime 50,000–100,000 valutazioni, seguito da un plateau di raffinamento graduale. La convergenza è più rapida per le istanze di set A e B (più facili), mentre per E-n101-k14 e P-n101-k4 (set E e P) l'algoritmo continua a migliorare fino alle fasi finali.

4.3 Visualizzazione delle Rotte Migliori

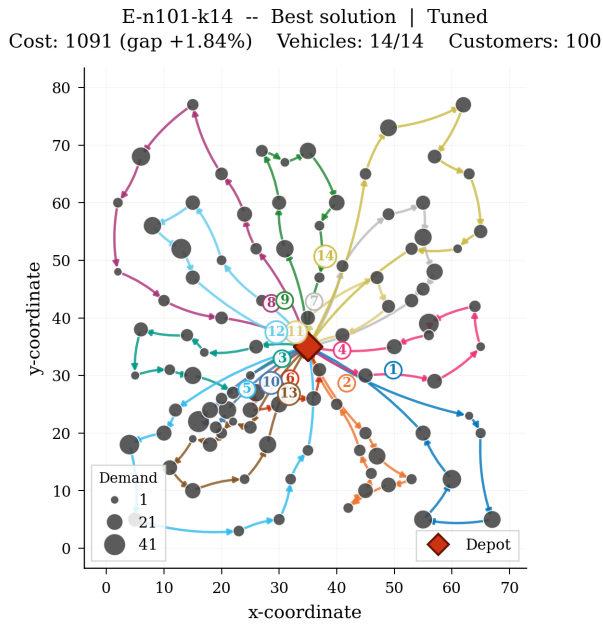
I percorsi della migliore soluzione trovata per ogni istanza rappresentativa sono visualizzati nello spazio 2D. Il depot è indicato con un rombo rosso, i clienti come cerchi proporzionali alla domanda.



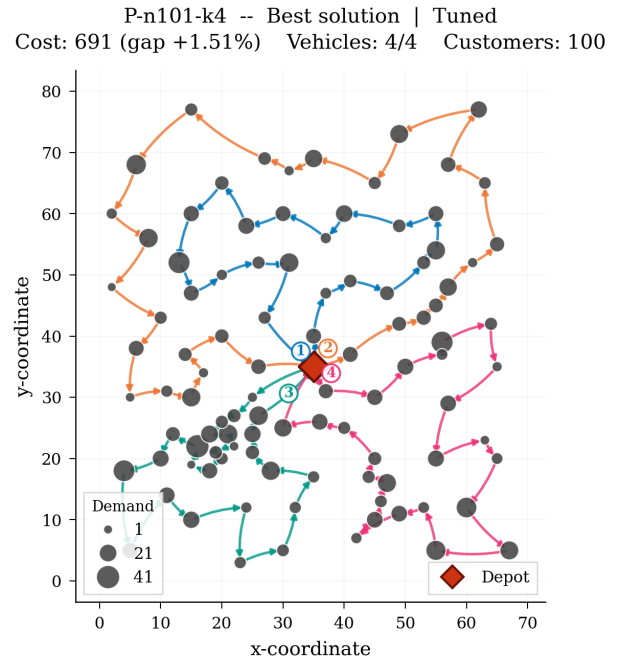
(a) Rotte per A-n45-k7



(b) Rotte per B-n66-k9



(c) Rotte per E-n101-k14



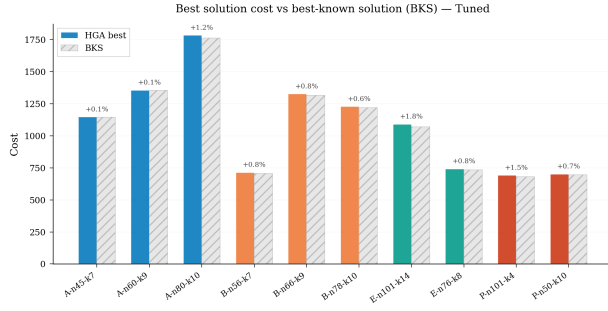
(d) Rotte per P-n101-k4

Figura 4.2: Visualizzazione grafica delle rotte finali — Configurazione Tuned.

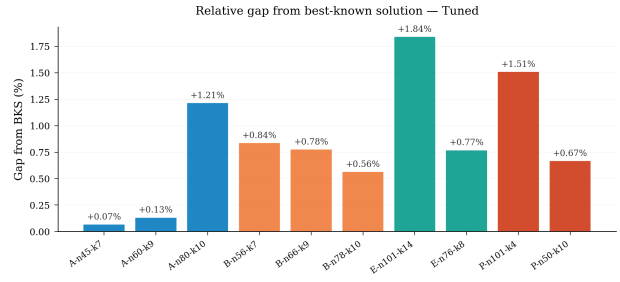
Le rotte appaiono logicamente coerenti: i clienti sono raggruppati in cluster geograficamente compatti, senza incroci evidenti tra i percorsi. L'istanza P-n101-k4 (solo 4 veicoli per 100 clienti) produce rotte molto lunghe ma geometricamente ben strutturate.

4.4 Analisi Statistica Globale

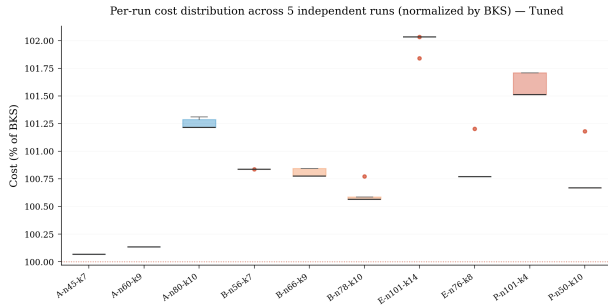
I grafici seguenti riassumono le prestazioni della configurazione Tuned su tutte le 10 istanze.



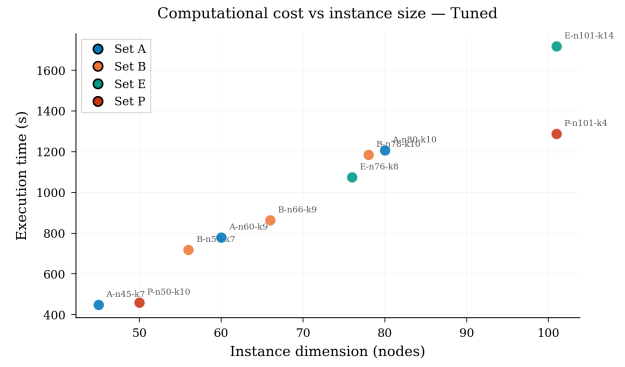
(a) Costi HGA vs BKS



(b) Gap percentuale dal BKS



(c) Distribuzione costi normalizzati



(d) Tempo di calcolo vs Dimensione

Figura 4.3: Statistiche globali sulle prestazioni e la stabilità — Configurazione Tuned.

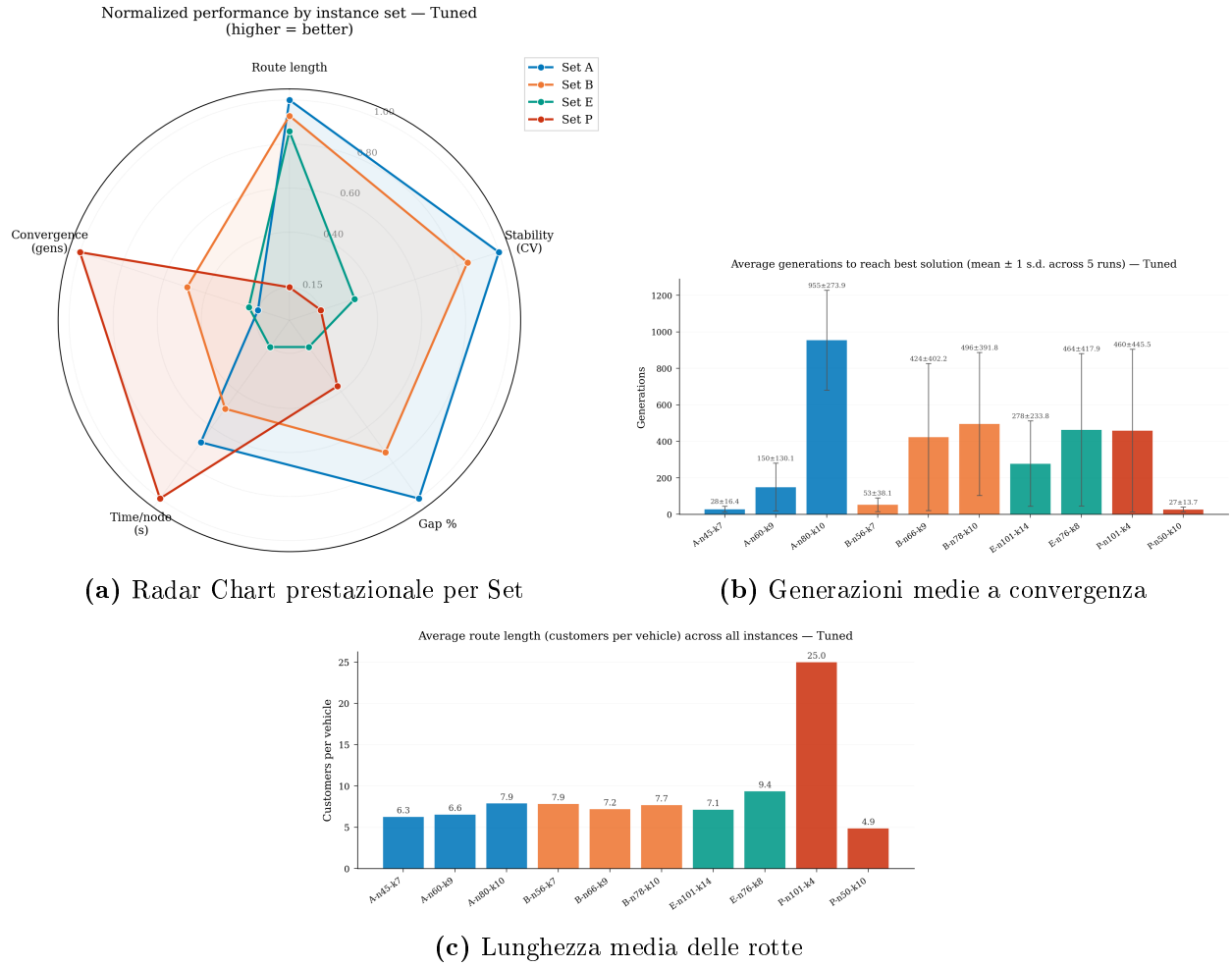


Figura 4.4: Prestazioni aggregate, convergenza evolutiva e struttura delle rotte.

4.5 Confronto tra Varianti di Configurazione

Per valutare l'impatto dei parametri, la configurazione Tuned è stata confrontata con sei varianti alternative, riassunte in Tabella 4.2.

Tabella 4.2: Varianti di configurazione a confronto

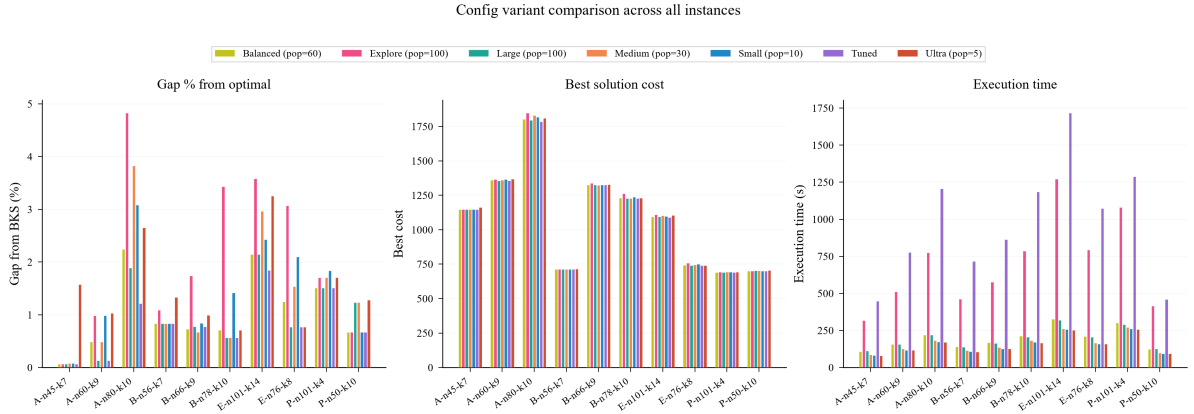
Variante	μ	k	e	γ	Generazioni
Tuned	81	4	4	25	$\sim 4,300$
Balanced	60	3	4	12	$\sim 5,800$
Large	100	4	5	15	$\sim 3,500$
Medium	30	3	3	7	$\sim 11,600$
Small	10	2	2	3	$\sim 35,000$
Fast	5	2	1	2	$\sim 70,000$

La tabella comparativa completa è riportata di seguito (Tabella 4.3), mentre il confronto visivo è mostrato in Figura 4.5.

Tabella 4.3: Config variant comparison — best solution cost and gap from BKS

Instance	Optimal	Tuned	Balanced	Explore	Large	Medium	Small	Fast
A-n45-k7	1,146	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.77 (+0.07%)	1,146.91 (+0.08%)	1,146.91 (+0.08%)	1,164.09 (+1.58%)
A-n60-k9	1,354	1,355.80 (+0.13%)	1,360.59 (+0.49%)	1,367.35 (+0.99%)	1,355.80 (+0.13%)	1,360.59 (+0.49%)	1,367.34 (+0.99%)	1,367.97 (+1.03%)
A-n80-k10	1,763	1,784.41 (+1.21%)	1,802.63 (+2.25%)	1,848.12 (+4.83%)	1,796.37 (+1.89%)	1,830.48 (+3.83%)	1,817.37 (+3.08%)	1,809.79 (+2.65%)
B-n56-k7	707	712.92 (+0.84%)	712.92 (+0.84%)	714.69 (+1.09%)	712.92 (+0.84%)	712.92 (+0.84%)	712.92 (+0.84%)	716.42 (+1.33%)
B-n66-k9	1,316	1,326.20 (+0.78%)	1,325.57 (+0.73%)	1,338.88 (+1.74%)	1,326.20 (+0.78%)	1,324.84 (+0.67%)	1,327.10 (+0.84%)	1,329.01 (+0.99%)
B-n78-k10	1,221	1,227.90 (+0.56%)	1,229.69 (+0.71%)	1,262.89 (+3.43%)	1,227.90 (+0.56%)	1,227.90 (+0.56%)	1,238.33 (+1.42%)	1,229.69 (+0.71%)
E-n101-k14	1,071	1,090.72 (+1.84%)	1,093.96 (+2.14%)	1,109.37 (+3.58%)	1,093.96 (+2.14%)	1,102.78 (+2.97%)	1,097.02 (+2.43%)	1,105.88 (+3.26%)
E-n76-k8	735	740.66 (+0.77%)	744.18 (+1.25%)	757.56 (+3.07%)	740.66 (+0.77%)	746.28 (+1.53%)	750.44 (+2.10%)	740.66 (+0.77%)
P-n101-k4	681	691.29 (+1.51%)	691.29 (+1.51%)	692.64 (+1.71%)	691.29 (+1.51%)	692.64 (+1.71%)	693.54 (+1.84%)	692.64 (+1.71%)
P-n50-k10	696	700.66 (+0.67%)	700.66 (+0.67%)	700.66 (+0.67%)	704.61 (+1.24%)	704.61 (+1.24%)	700.66 (+0.67%)	704.91 (+1.28%)

Tuned: pop=81, tournament=4, elite=4, granular=25; **Balanced:** pop=60, tournament=3, elite=4, granular=12; **Explore:** pop=100, tournament=2, elite=1, granular=15; **Large:** pop=100, tournament=4, elite=5, granular=15; **Medium:** pop=30, tournament=3, elite=3, granular=7; **Small:** pop=10, tournament=2, elite=2, granular=3; **Fast:** pop=5, tournament=2, elite=1, granular=2

**Figura 4.5:** Confronto visivo tra le sette varianti di configurazione: gap %, costo migliore e tempo di esecuzione.

4.6 Discussione dei Risultati

L'analisi dei risultati permette di trarre diverse osservazioni:

- **Qualità delle soluzioni:** La configurazione Tuned ($\mu = 81$) ottiene risultati complessivamente eccellenti, con un gap medio dello 0.84%. Rispetto alla configurazione Large ($\mu = 100$, gap medio $\sim 1.0\%$), Tuned raggiunge prestazioni comparabili o superiori con una popolazione più piccola, grazie ai parametri finemente calibrati da Optuna (crossover 0.675 anziché 0.8, granular size 25 anziché 15).
- **Effetto della dimensione della popolazione:** Riducendo μ da 100 (Large) a 5 (Fast), il gap medio peggiora progressivamente (Large: $\sim 1.0\%$, Fast: $\sim 1.7\%$). Il degrado è particolarmente evidente su A-n45-k7 (da 0.07% a 1.58%) e A-n80-k10. Tuned e Balanced, con popolazioni intermedie (81 e 60), rappresentano il miglior compromesso tra diversità genetica e numero di generazioni.
- **Robustezza:** La deviazione standard è generalmente contenuta in tutte le configurazioni, indicando che l'algoritmo è intrinsecamente stabile. La configurazione Tuned mostra la variabilità minore, con $\sigma < 1$ su 7 istanze su 10.
- **Velocità di convergenza:** L'ibridazione con ricerca locale accelera la convergenza iniziale in tutte le configurazioni. Tuned, con $\gamma = 25$, beneficia di una ricerca locale

granulare più ampia che contribuisce ai risultati superiori, esplorando un vicinato più esteso senza il costo computazionale della ricerca esaustiva.

- **Trade-off qualità/tempo:** Le configurazioni con popolazione minore sono più veloci (meno tempo per generazione) ma richiedono più generazioni per convergere. Il tempo totale è dominato dal numero di valutazioni FE (fissato a 350,000), quindi tutte le configurazioni hanno tempi comparabili.

Capitolo 5

Conclusioni

Il progetto ha dimostrato l'efficacia dell'Hybrid Genetic Algorithm per la risoluzione del Capacitated Vehicle Routing Problem. L'approccio ibrido, che combina la potenza esplorativa degli algoritmi genetici con l'efficacia della ricerca locale, produce soluzioni di alta qualità sulle istanze benchmark CVRPLIB.

5.1 Scelte Progettuali e Originalità

5.1.1 Scelte Chiave

1. **Split Algorithm** [4]: L'uso dello split di Prins come decodifica è fondamentale. Invece di codificare esplicitamente le route nel cromosoma, si lascia che la DP trovi la partizione ottimale, riducendo lo spazio di ricerca e garantendo l'ottimalità della decodifica data la permutazione.
2. **Ricerca Locale Granulare**: L'integrazione della GLS con $\gamma = 25$ permette di esplorare un vicinato inter-route molto più ampio rispetto alle configurazioni standard ($\gamma = 15$), senza il costo computazionale proibitivo della ricerca esaustiva $O(N^2)$.
3. **Ottimizzazione Iperparametrica**: L'uso di Optuna per il tuning automatico dei parametri ha permesso di scoprire una configurazione non ovvia ($\mu = 81$, $p_c = 0.675$, $p_m = 0.236$, $p_{ls} = 0.259$) che supera le configurazioni progettate manualmente.
4. **Accelerazione Numba**: L'uso del compilatore JIT Numba per i calcoli critici (matrici di distanza, Split DP, operatori di ricerca locale) ha permesso di ottenere performance computazionali elevate in Python puro, senza ricorrere a implementazioni in C/C++.
5. **Architettura client-server con WebSocket**: L'algoritmo è integrato in un'architettura moderna con backend Python (FastAPI) e frontend React, permettendo la visualizzazione in tempo reale dell'evoluzione dell'algoritmo tramite WebSocket.

5.1.2 Difficoltà Affrontate

- **Vincoli di capacità**: Gestire i vincoli di capacità durante il crossover e la mutazione è complesso perché questi operatori lavorano sulla permutazione, non sulle route. Lo split algorithm risolve elegantemente questo problema.
- **Bilanciamento esplorazione-sfruttamento**: Trovare il giusto tasso di ricerca locale è stato cruciale: troppo alto causa convergenza prematura, troppo basso rallenta il

miglioramento. Il tuning con Optuna ha permesso di individuare il valore ottimale $p_{ls} = 0.259$.

- **Scalabilità:** Su istanze con 101 nodi, lo split $O(n^2)$ e la ricerca locale diventano computazionalmente significativi, richiedendo attenzione all’efficienza implementativa e all’uso di Numba JIT.

5.2 Risultati Raggiunti

I punti di forza dell’implementazione includono:

- L’uso dello split algorithm per una decodifica ottimale;
- Un insieme diversificato di operatori genetici e di ricerca locale;
- L’ottimizzazione iperparametrica con Optuna che ha prodotto la configurazione Tuned;
- Un’architettura software moderna con visualizzazione in tempo reale;
- Robustezza e consistenza dei risultati tra run diverse;
- Gap medio dello 0.84% sul benchmark CVRPLIB, con prestazioni eccellenti sulle istanze di set A e B.

5.3 Sviluppi Futuri

Possibili estensioni del lavoro includono:

- L’implementazione di una ricerca locale più aggressiva (Variable Neighborhood Search) [5];
- L’uso di tecniche di diversificazione della popolazione (crowding, fitness sharing);
- L’estensione ad altre varianti del VRP (VRPTW con finestre temporali, VRPPD con pickup and delivery);
- L’implementazione di un algoritmo memetico con ricombinazione ottimale dei percorsi (edge assembly crossover) [17];
- L’integrazione con un solver esatto per il post-processing delle route (set partitioning).

Il codice sorgente del progetto, comprensivo di backend Python, frontend React e relazione L^AT_EX, è disponibile nella repository del progetto.

Bibliografia

- [1] George B. Dantzig e John H. Ramser. «The Truck Dispatching Problem». In: *Management Science* 6.1 (1959), pp. 80–91. DOI: 10.1287/mnsc.6.1.80.
- [2] Paolo Toth e Daniele Vigo, cur. *The Vehicle Routing Problem*. Society for Industrial e Applied Mathematics (SIAM), 2002. ISBN: 978-0-89871-579-8.
- [3] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989. ISBN: 978-0-201-15767-3.
- [4] Christian Prins. «A simple and effective evolutionary algorithm for the vehicle routing problem». In: *Computers & Operations Research* 31.12 (2004), pp. 1985–2002. DOI: 10.1016/S0305-0548(03)00158-8.
- [5] Thibaut Vidal et al. «A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems». In: *Operations Research* 60.3 (2012), pp. 611–624. DOI: 10.1287/opre.1120.1048.
- [6] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975. ISBN: 978-0-262-58111-0.
- [7] Shen Lin e Brian W. Kernighan. «An Effective Heuristic Algorithm for the Traveling-Salesman Problem». In: *Operations Research* 21.2 (1973), pp. 498–516. DOI: 10.1287/opre.21.2.498.
- [8] Ilhan Or. «Traveling Salesman-Type Combinatorial Problems and their Relation to the Logistics of Regional Blood Banking». Tesi di dott. Northwestern University, 1976.
- [9] Geoff Clarke e John W. Wright. «Scheduling of Vehicles from a Central Depot to a Number of Delivery Points». In: *Operations Research* 12.4 (1964), pp. 568–581. DOI: 10.1287/opre.12.4.568.
- [10] David E. Goldberg e Kalyanmoy Deb. «A Comparative Analysis of Selection Schemes Used in Genetic Algorithms». In: *Foundations of Genetic Algorithms*. Vol. 1. Morgan Kaufmann, 1991, pp. 69–93.
- [11] Lawrence Davis. «Applying Adaptive Algorithms to Epistatic Domains». In: *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI)*. 1985, pp. 162–164.
- [12] Paolo Toth e Daniele Vigo. «The Granular Tabu Search and Its Application to the Vehicle-Routing Problem». In: *INFORMS Journal on Computing* 15.4 (2003), pp. 333–346. DOI: 10.1287/ijoc.15.4.333.24890.
- [13] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Accessed: 2025-06-01.
- [14] Siu Kwan Lam, Antoine Pitrou e Stanley Seibert. «Numba: A LLVM-Based Python JIT Compiler». In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*. 2015, pp. 1–6. DOI: 10.1145/2833157.2833162.

- [15] CVRPLIB. *Capacitated Vehicle Routing Problem Library*. <http://vrp.atd-lab.inf.puc-rio.br/>. Accessed: 2025-06-01.
- [16] Takuya Akiba et al. «Optuna: A Next-Generation Hyperparameter Optimization Framework». In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.
- [17] Thibaut Vidal et al. «A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows». In: *Computers & Operations Research* 40.1 (2013), pp. 475–489. DOI: 10.1016/j.cor.2012.07.018.